M09 Learner Book

ETL/ELT + Data Quality + Transformation Engineering | Release Candidate 1 | 13 controlled lessons

M09-L01 - ETL vs ELT and transformation boundaries

Learning objective

Choose where transformations run based on contracts, platform capabilities and governance.

Why this matters in Data Engineering

ETL and ELT are execution patterns, not competing religions; both need tests, lineage and recoverability.

Understand

• ETL transforms before loading a serving platform.

• ELT lands/loads raw data then transforms inside the data platform.

• Layer boundaries should preserve source evidence and make failures diagnosable.

• Choose based on source limits, platform scale, governance, latency and cost.

Try / inspect

```
ETL: source -> Python transform -> serving
ELT: source -> raw platform -> SQL/dbt -> serving
```

Common failure

Selecting ETL/ELT from fashion instead of defining where raw evidence, quality checks and recovery live.

Your turn - independent proof

Map the Stage 09 project operations to ETL-like boundaries, then describe the Stage 10/warehouse ELT alternative.

Move on when

You can justify transformation placement and state what evidence remains invariant across both patterns.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M09-L02 - Raw, staging, clean/intermediate and serving contracts

Learning objective

Give each layer a narrow responsibility and prohibited behavior.

Why this matters in Data Engineering

Clear contracts prevent a transformation project from becoming one opaque script/table.

Understand

• Raw preserves source evidence.

• Staging casts/renames/normalizes close to source.

• Intermediate/clean logic implements reusable business rules.

• Serving exposes stable grain/measures for consumers; quarantine is separate.

Try / inspect

```
RAW -> STAGING -> INTERMEDIATE -> SERVING
          bad -> QUARANTINE
```

Common failure

Mixing destructive cleaning into raw or burying all business logic in one serving query.

Your turn - independent proof

Write one-sentence allowed/not-allowed rules for every project layer and inspect the folder/model layout against them.

Move on when

You can place a new transformation in a layer and defend the placement.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M09-L03 - Deterministic transformations and production SQL style

Learning objective

Make the same valid input produce the same business result with explicit tie-breakers and projections.

Why this matters in Data Engineering

Nondeterministic row selection can silently change outputs between runs.

Understand

• Trim/case/type normalization must be explicit.

• Use stable tie-breakers for latest-row logic.

• Avoid SELECT * in durable contracts.

• Separate complex steps so grain changes are visible.

Try / inspect

```
row_number() over (partition by order_id order by updated_at desc, source_seq desc)
```

Common failure

Using max(updated_at) without a deterministic tie-breaker when source rows can tie.

Your turn - independent proof

Write deterministic latest-row logic and compare it with an intentionally unsafe tied-timestamp version.

Move on when

Repeated runs on unchanged input return identical rows/values and the grain is explicit.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M09-L04 - Deduplication and key integrity

Learning objective

Resolve duplicate business keys only with a declared winner/ambiguity policy.

Why this matters in Data Engineering

Dropping duplicates is a business decision disguised as data cleaning unless the identity/order rule is explicit.

Understand

• Business key defines logical identity.

• Latest-wins needs deterministic ordering.

• Conflicting equal-latest records should quarantine rather than guess.

• Count duplicates discarded/quarantined in run metadata.

Try / inspect

```
O6101: 08:00 then 09:00 -> keep 09:00
O6102: two conflicting 08:30 -> quarantine ambiguity
```

Common failure

Calling drop_duplicates() without documenting which record survives and why.

Your turn - independent proof

Run the duplicate batch and account for every source row as staged, discarded or quarantined.

Move on when

Every duplicate outcome follows a deterministic rule and row accounting reconciles.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M09-L05 - Schema checks and data contracts

Learning objective

Block incompatible structure/type changes before they corrupt serving state.

Why this matters in Data Engineering

A pipeline that silently adapts to wrong columns can be more dangerous than one that fails loudly.

Understand

• Define required columns/types/allowed values.

• Classify additive compatible drift vs breaking rename/removal/type change.

• Validate whole-batch contract before serving write/state advance.

• Keep failed schema evidence and do not weaken tests to pass.

Try / inspect

```
expected_cols={order_id,updated_at,customer_id,region,product,qty,unit_price,discount_pct,currency}
```

Common failure

Renaming sales_region to region automatically without an approved mapping and history.

Your turn - independent proof

Run batch_schema_drift and prove serving rows/state remain unchanged after failure.

Move on when

Breaking drift is detected before target mutation and failure evidence is preserved.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M09-L06 - Source-to-target reconciliation

Learning objective

Prove every source row/key/value has an explainable target outcome.

Why this matters in Data Engineering

Row counts alone can hide missing/duplicated keys or wrong values after upserts.

Understand

• Row accounting: source = staged + quarantine + explicitly discarded.

• Every staged key should exist exactly once in serving unless policy says otherwise.

• Applied target values must match the accepted staged version.

• Whole-table totals complement, not replace, key-level reconciliation.

Try / inspect

```
source_rows = staged_rows + quarantine_rows + dedup_discarded
# staged keys == target keys for applied records
```

Common failure

Declaring success because the serving count looks reasonable without checking key/value equivalence.

Your turn - independent proof

Hand-calculate batch_001 reconciliation then compare the run manifest and serving rows.

Move on when

Counts, keys and selected measures reconcile independently.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M09-L07 - Incremental processing and safe reruns

Learning objective

Apply only new/changed records while keeping reruns idempotent.

Why this matters in Data Engineering

Incremental logic reduces repeated work but adds state/order complexity and replay risk.

Understand

• Stable keys define insert/update identity.

• Source/file identity can prevent accidental reprocessing.

• Same/stale versions must be ignored explicitly, not overwrite newer target state.

• Replay mode should keep final business state stable.

Try / inspect

```
batch_001 -> inserts
batch_002 -> inserts + update
same file -> skip; --replay -> stable unique keys
```

Common failure

Using ingestion timestamp as winner when business update timestamp defines the source version.

Your turn - independent proof

Run batch_001, batch_002, identical rerun and replay; prove unique keys and expected O6002 update.

Move on when

Final serving state is stable across normal rerun/replay and all decisions are counted.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M09-L08 - Failure behavior, retries, quarantine and atomic state

Learning objective

Fail loudly without false progress and separate recoverable row defects from blocking failures.

Why this matters in Data Engineering

A failed run must never mark source state complete or publish partial trusted outputs as success.

Understand

• Quarantine handles record-level defects with reasons.

• Quality thresholds may block serving publication.

• Retry only transient external operations; deterministic transform defects need code/data repair.

• Commit processed-state only after successful publish/reconciliation.

Try / inspect

```
try: validate -> transform -> publish -> reconcile -> commit_state
except: preserve evidence; state unchanged
```

Common failure

Writing processed-file state before target/reconciliation finishes.

Your turn - independent proof

Inject a bad-quality batch, prove failure leaves state/serving unchanged, repair and rerun.

Move on when

Failure evidence is visible and recovery does not duplicate or skip business records.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M09-L09 - Lineage, run metadata and auditability

Learning objective

Connect source batch, code/version, transformation outputs and target evidence through a run ID.

Why this matters in Data Engineering

When a number is questioned, you need to reconstruct which input and logic produced it.

Understand

• Run metadata should include input identity, code/config version and start/end/status.

• Lineage connects models/steps, not only files.

• Keep logs structured enough to correlate run IDs and counts.

• Do not put secrets or unnecessary sensitive data in evidence.

Try / inspect

```
run_id -> source_sha -> staged_count -> model/build -> serving_count -> checkpoint
```

Common failure

Keeping only final CSV/table with no record of which source or transformation version produced it.

Your turn - independent proof

Add run_id/source SHA/code revision fields to the manifest and trace one serving row back to source.

Move on when

A reviewer can reconstruct the pipeline lineage from saved evidence.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M09-L10 - Build and defend a reliable batch pipeline

Learning objective

Integrate contracts, transforms, quality, serving, state and recovery into one independent run.

Why this matters in Data Engineering

The module standard is production behavior across failure/rerun conditions, not passing isolated examples.

Understand

• Reset outputs intentionally before first run.

• Operate normal + incremental + failure + replay scenarios.

• Save manifests/quarantine/serving/validation evidence.

• Explain trade-offs and known limitations in a runbook.

Try / inspect

```
python -m src.pipeline_TODO source/batch_001.csv
# inspect outputs, then incremental/failure/replay
```

Common failure

Finishing the happy path and skipping failure/replay validation.

Your turn - independent proof

Complete DE09-10 closed-book on the supplied project and defend every row-accounting/state decision.

Move on when

You can operate and explain the full scenario without the reference implementation open.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M09-L11 - dbt-style project anatomy, sources and dependency graph

Learning objective

Express transformation dependencies and source contracts explicitly in a model project.

Why this matters in Data Engineering

Transformation code scales better when models/tests/docs form a visible DAG instead of hard-coded cross-table references.

Understand

• source() declares upstream raw objects; ref() declares model dependencies.

• Project/model/test configuration belongs in version control.

• Staging models should remain source-close.

• A dependency graph supports ordered builds, lineage and impact reasoning.

Try / inspect

```
select * from {{ ref("stg_orders") }}
# source("raw","orders") for declared upstream
```

Common failure

Hard-coding downstream relation names so lineage/build order becomes hidden.

Your turn - independent proof

Create a small model DAG map from raw -> staging -> intermediate -> mart and identify every dependency edge.

Move on when

A reviewer can follow dependencies without reading every SQL file.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M09-L12 - Executable data tests, quality severity and CI boundaries

Learning objective

Turn data contracts into repeatable checks that block unsafe publication.

Why this matters in Data Engineering

Manual spot checks do not scale; tests make expectations executable and reviewable.

Understand

• Use uniqueness/not-null/relationship/accepted-value tests where appropriate.

• Add singular/custom assertions for business invariants.

• Distinguish blocking errors from warnings intentionally.

• CI should fail before publishing when critical tests fail.

Try / inspect

```
tests: unique(order_id), not_null(customer_id), relationships(customer_id), accepted_values(status)
custom: paid_amount > 0
```

Common failure

Changing a failing test to warning because the data is inconvenient.

Your turn - independent proof

Add one key, one relationship and one custom invariant test; inject a bad row and prove the build blocks.

Move on when

Critical contract violations stop the build and produce actionable evidence.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M09-L13 - Incremental models, materialization, documentation and handoff

Learning objective

Choose durable model behavior and package the transformation project for repeatable operation.

Why this matters in Data Engineering

Views/tables/incremental models have different cost/freshness/rebuild trade-offs; handoff must state how to rebuild and recover.

Understand

• Choose view/table/incremental based on reuse, cost, freshness and change handling.

• Incremental unique_key/filter/schema-change policy must be explicit.

• Document grain, owner, freshness, tests and known limitations.

• A clean environment should reproduce build/test from recorded dependencies.

Try / inspect

```
checkout -> install -> setup test data -> build/tests -> preserve logs
HANDOFF.md: grains, rerun/full-refresh, limitations
```

Common failure

An incremental model that works on first run but duplicates/loses rows on update/full refresh.

Your turn - independent proof

Design one incremental serving model and a clean-run handoff; explain full-refresh/rebuild implications.

Move on when

Another engineer can rebuild, test and safely rerun the transformation project.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.